Backup, Restore, and Recovery

The database is running.
Yesterday's records are gone.

Day 1: recover a small PostgreSQL database
Day 2: connect recovery to the midterm case

An accidental committed deletion

```
BEGIN;
DELETE FROM metro_support.ticket_events;
COMMIT;
```

Illustrative incident: the operator intended to remove one test event.
The omitted WHERE condition removes every event.

A restart preserves the committed change. A replica may copy it.
Recovery needs an earlier usable state.

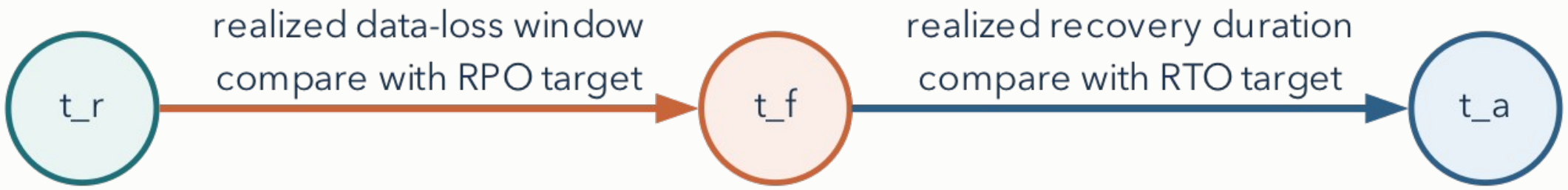
Availability and recovery cover different failures

Mechanism	What it provides	Important limit
High availability	Continued service after some component failures	Can reproduce a harmful committed write
Backup	A retained recoverable state	Must remain accessible and restorable
Restore	Reconstruction into a target	Needs verification before application use
Disaster recovery	A tested service-recovery procedure	Includes people, access, systems, and dependencies

The failure you plan for determines what you must keep and test.

Recovery point and recovery time objectives

RPO: acceptable data-loss window.
RTO: target duration to restore acceptable service.



`t_r: newest recoverable state` `t_f: failure` `t_a: service acceptable`

```
Data-loss window = t_f − t_r
Service-recovery duration = t_a − t_f
```

A worked recovery timeline

Time	Event
14:00	Newest usable export represents this state
14:17	Harmful deletion commits
14:21	Operator recognizes the incident
14:29	Restore and verification finish; service is acceptable

Data gap: 17 minutes. Recovery duration: 12 minutes.

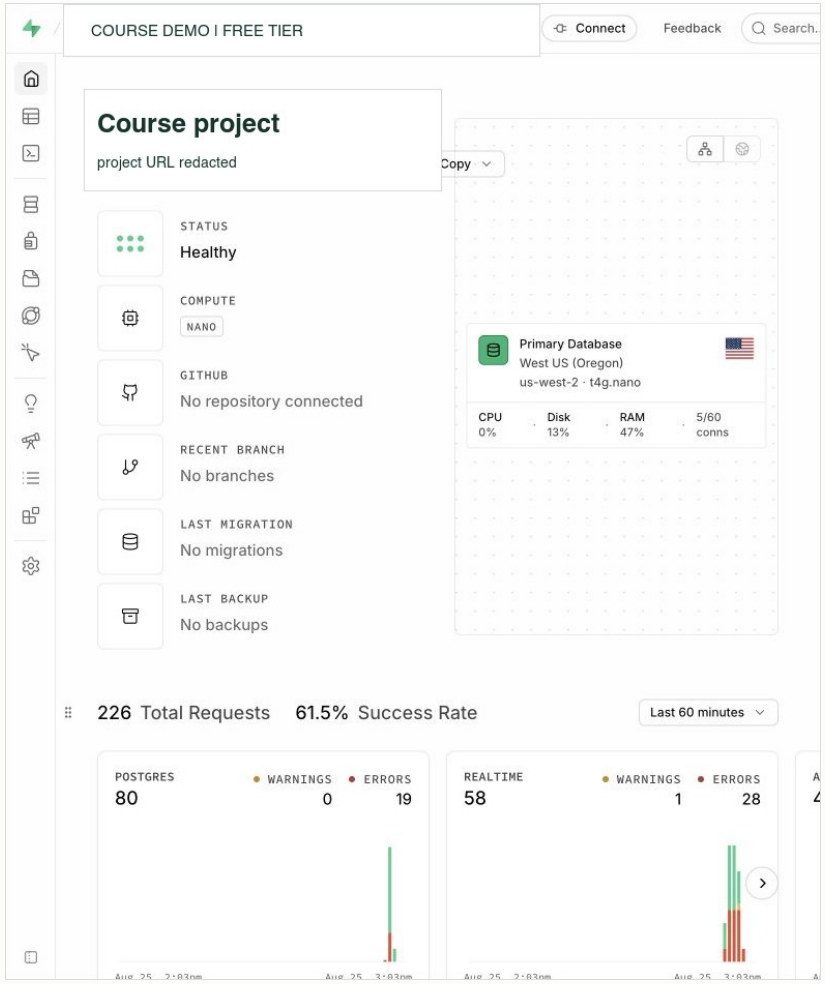
An RPO of 5 minutes and an RTO of 10 minutes would both be missed.
Illustrative scenario. Time gaps do not give the number of lost records.

Logical and physical recovery

Approach	What it reconstructs	What it depends on
Logical export	Objects and rows through database commands	Compatible tools, included dependencies, restore time
Physical backup and logs	Database storage and a supported history of changes	Compatible base backup and retained log sequence

Today: pg_dump and pg_restore for one course schema.
CSV alone does not preserve its keys, checks, indexes, or grants.

Supabase project health and retained backups



Healthy describes present availability.

No backups describes the displayed recovery history.

Supabase Free lacks the automatic backups offered on paid plans. Our lab uses a logical dump and a separate local restore.

Redacted course capture: August 25, 2026. Plan documentation checked September 7, 2026.

A small relational database for the restore

users

3 rows
user_id PK

tickets

3 rows
ticket_id PK
requester_id FK

ticket_events

5 rows
event_id PK
ticket_id FK

One user can request many tickets. One ticket can have many events.

Notebook fixture: 3 users / 3 tickets / 5 events.
Full Metro Support fixture elsewhere: 8 / 12 / 21. Compare the correct baseline.

Python launches tools; psql sends SQL

```
subprocess.run(
    PG_PREFIX + [PSQL, '-X', '-d', SOURCE_DB,
                 '-c', 'SELECT count(*) FROM metro_support.tickets;'],
    check=True, text=True, capture_output=True,
)
```

Python waits for psql to finish.
psql connects to SOURCE_DB and submits the SELECT.
PostgreSQL evaluates the SQL and returns the count.

In Colab, PG_PREFIX runs the local command as the postgres operating-system user.

The custom archive contains one schema

```
pg_dump --format=custom \
  --schema=metro_support --no-privileges \
  --file=metro_support.dump \
  --dbname="$SOURCE_DB"
```

Shell form of the notebook operation. SOURCE_DB is a database name, not a password-bearing URL. The notebook supplies the unique name.

Custom archive: --no-owner belongs on pg_restore.
Roles and cloud project settings need separate recovery planning.

Archive inspection and a checksum

```
pg_restore --list metro_support.dump
```

Check	What it establishes	What remains untested
Archive contents	Expected object inventory appears	Whether restoration will succeed
File size	Bytes exist	Completeness or correctness
SHA-256 comparison	Whether bytes match a trusted earlier hash	Whether the original content was trustworthy

A dump can execute SQL chosen by its source. Only use your own course archive in this lab.

A separate empty database receives the restore

```
createdb "$RESTORE_DB"
pg_restore --exit-on-error --single-transaction \
  --no-owner --no-privileges \
  --dbname="$RESTORE_DB" metro_support.dump
```

Stop on failure. Keep this small restore transactional.
Review ownership and grants separately before application use.

SOURCE_DB and RESTORE_DB are different databases on one temporary server.
They still share the risk of losing the runtime.

The source baseline gives specific expected results

Check	Source	Restored target
users / tickets / ticket_events	3 / 3 / 5	3 / 3 / 5
Tickets with no matching requester	0	0
in_progress / open / resolved	1 / 1 / 1	1 / 1 / 1
Named status constraint	tickets_status_allowed	tickets_status_allowed

The notebook compares complete results, including table names.
Equal counts alone could conceal a changed subject or requester.

A known-ticket query checks identity and values

```
SELECT t.ticket_id, t.status, t.subject,
       u.display_name AS requester
FROM metro_support.tickets AS t
JOIN metro_support.users AS u ON u.user_id = t.requester_id
WHERE t.ticket_id = 1001;
```

1001, open, Streetlight dark near bus stop, Requester 101

Changing only the subject leaves the row counts unchanged.
Compare values with the source and the intended original record.

A restored constraint must reject the intended violation

```
BEGIN;
INSERT INTO metro_support.tickets
    (ticket_id, requester_id, status, subject)
VALUES (1099, 101, 'almost_done', 'Constraint restore test');
ROLLBACK;
```

Expected: SQLSTATE 23514 and tickets_status_allowed

A missing table or disconnected server is a different failure.
The notebook also checks that test ticket 1099 was not retained.

Lab: recover and check one ticket of your own

Run the recovery notebook in Colab.

Adapt its known-ticket query to ticket 1002 or 1003.

Explain what that check adds and what remains untested.

Keep the output and short recovery account in the notebook.
Then remove the practice resources with the final cleanup cell.

Individual work. Brightspace: one completed notebook, no second report.

Day 2: the PostgreSQL operations clinic

A working schema must survive ordinary changes and mistakes.

Review: transactions, blocking, and recovery choices
Work: your individual midterm operations case

A small schema change can be rehearsed in a transaction

```
BEGIN;
ALTER TABLE metro_support.tickets ADD COLUMN follow_up text;
SELECT column_name FROM information_schema.columns
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets' AND column_name = 'follow_up';
ROLLBACK;
```

Inside the transaction: follow_up exists.
After ROLLBACK: the same catalog query returns no rows.

Disposable notebook fixture only. A transactional change can still take locks.

Rollback, forward repair, and restore

Situation	Candidate response	Decision constraint
Harmful write remains uncommitted	ROLLBACK the owning transaction	Coordinate with the transaction owner
Small committed mistake with a known correction	Reviewed compensating change	Avoid overwriting valid newer work
Committed loss requiring an earlier state	Separate restore and verified recovery	Reconcile later valid changes before cutover

A database backup does not make every incident a restore incident.

A waiting writer may be a transaction problem

Session	Observed state in the Week 5 exercise
A	Changed a ticket and left its transaction open
B	Waiting to update the same ticket
Diagnostic connection	pg_blocking_pids identifies A as B's blocker

After the owner resolves A, B can finish.
Verify the final row and close the practice connections.

This episode does not require restoring yesterday's database.

An improvement starts with a specific operating problem

Problem	Possible improvement	A useful comparison
Reader can write data	Least-privilege role	Allowed SELECT and denied write
User can see another user's rows	Row-level access policy	Two identities with different visible rows
Small queue requires a broad scan	Query-appropriate index	Same result IDs, different plan work
Application needs an additional field	Safe schema change	Old behavior still works after the change

The midterm offers these alternatives. Choose one relevant to your case.

A recovery plan identifies the target and acceptance checks

Purpose: recover the Metro Support schema after accidental loss.

Target: an approved empty database, separate from the source.

Checks: expected objects and counts, a known ticket, a report,
and the intended constraint failure.

Before application use: restore reviewed access and verify a safe client path.

Midterm: a recovery plan is required. An executed restore is optional extra work.
Keep planned checks distinct from checks you actually ran.

A concise handoff explains the recovery result

I restored the three-table practice archive into a separate database. The counts and grouped status report matched the source. Ticket 1001 retained its subject and requester. The restored status rule rejected the invalid test value. I have not restored application roles or tested application reconnection, so this drill does not establish that the service is ready for traffic.

Worked example from the notebook. Use your own results and limitations when explaining your midterm case.

Lab 2: your midterm, in your own Supabase project

Step	What you do	Part	You should see	Min
0	Connect through the **session pooler**; load the starter	Setup	8 users, 12 tickets, 21 events	10
1	Fill in the join; test two rules with writes that should fail	Part 1	12 rows; SQLSTATE 23502 and 23514	20
2	Run the rollback, then write the commit	Part 2	low, high, low · low, medium, medium	15
3	Index experiment, before and after	Part 4	1011, 1007, 1001; about 16 kB	15
4	Back up Supabase, restore in Colab, compare	Part 5	every check says **MATCH**	15
5	Write where you are, then clean up			5

Classwork, not the midterm. Submit this notebook today. Copy SQL into `midterm.sql` and results into `README.md` as you go. The midterm is due **Tuesday, November 3**.

IPv4 and IPv6: why Colab needs the session pooler

IPv4

32-bit address, four numbers

`203.0.113.25`

About 4.3 billion addresses. The world ran out.

IPv6

128-bit address, hexadecimal groups

`2001:db8::25`

Enough addresses for every device.

Colab runtime
reaches IPv4 only

✗ **no route**

Direct connection
db.[ref].supabase.co · IPv6 on the Free plan

Colab runtime
reaches IPv4 only

✓ **IPv4** →

Session pooler
accepts IPv4

→

Your database
midterm_metro

`Network is unreachable` usually means you copied the **direct** connection. Copy the **session pooler** instead.

A connection pooler lends out database connections

A **pooler** sits between clients and PostgreSQL. It keeps a few real database connections open and lends them to clients.

Session mode · port 5432

- You keep **one connection** until you disconnect
- `BEGIN` … `COMMIT`, `SET`, and temp tables work
- `pg_dump` gets one consistent session
- Supabase: **port 5432** on the pooler host

Transaction mode · port 6543

- You get a connection for **one transaction**, then it returns to the pool
- Session settings can vanish between transactions
- No prepared statements
- Made for serverless functions with many short connections

Today: **session mode**. Your transaction tests and your backup need **one connection for the whole session**.

Three ways into the same Supabase database

Option	Host and port	User	Network	Use it for
Direct	`db.[ref].supabase.co:5432`	`postgres`	IPv6 (IPv4 is a paid add-on)	pg_dump when your network has IPv6
Session pooler	`aws-[n]-[region].pooler.supabase.com:5432`	`postgres.[ref]`	IPv4	**Today, from Colab**
Transaction pooler	same host, port 6543	`postgres.[ref]`	IPv4	serverless apps with many short connections

> Find it: your project › **Connect** › **Session pooler**. Copy the string from there; the `aws-[n]` part cannot be guessed from your region.

Your connection string, piece by piece

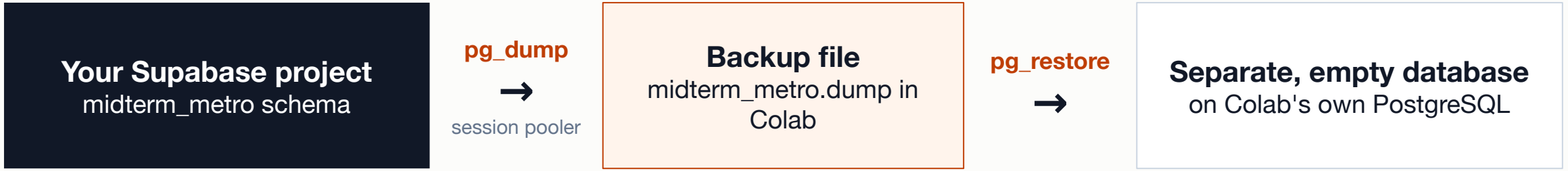
```
postgresql://postgres.[ref]:[YOUR-PASSWORD]@aws-[n]-[region].pooler.supabase.com:5432/postgres
```

Piece	In your string
Protocol	`postgresql://`
User	`postgres.[ref]`
Password	`[YOUR-PASSWORD]`: replace it
Host	`aws-[n]-[region].pooler.supabase.com`
Port	`5432`, the session pooler
Database	`postgres`

Keep it safe

- Paste it at the notebook's **hidden prompt** only. Never in a cell, your README, or a screenshot.
- Encode special characters: `@` → `%40`, `#` → `%23`, `/` → `%2F`.
- `sslmode=require` **encrypts** the connection. It does not verify the server.
- Forgot it? Reset it under **Project Settings** › **Database**.

Part 5 for real: back up Supabase, restore in Colab



`pg_dump` must be the **same major version** as your server, or newer. The notebook installs matching tools.

Same checks on both sides: counts · join rows · ticket 1004 · index. Every line should say **MATCH**.

> **Executed, not just planned.** If your midterm database is this Supabase project, report this restore as **observed**: the date, source, target, commands, and each check's result.
>
> **Limits:** roles and grants were left out, and the Colab copy disappears when the runtime ends. A real backup is kept somewhere that outlives the machine that made it.

Clinic resources and the next step

Chapter 8: recovery commands and the runbook example

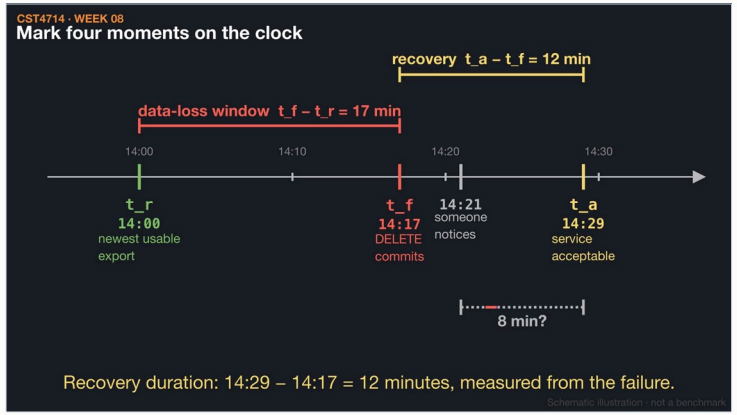
Week 5 notebook: the blocking experiment

Canonical midterm assignment: requirements and rubric

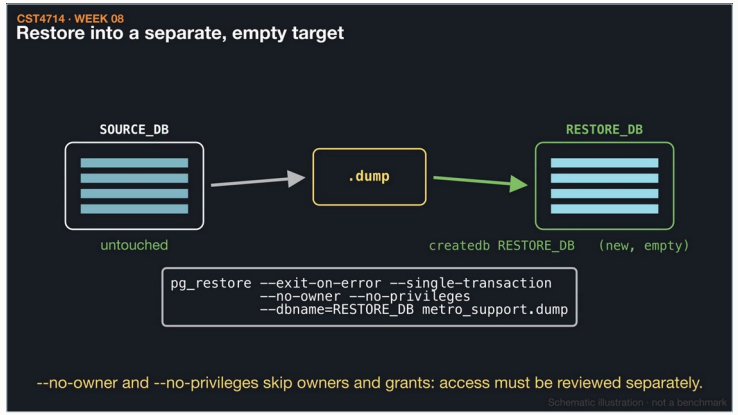
After Lab 2, use any remaining time to repair one incomplete part, rerun it, and update the explanation beside the result.

Next week: NoSQL models and JSON, compared with the relational strengths you now know.

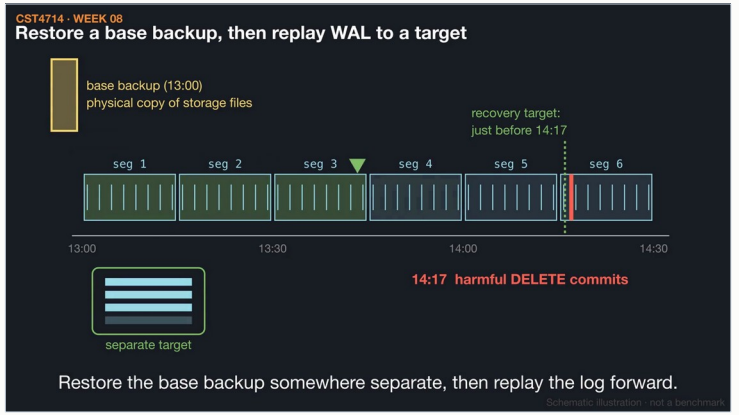
This week's videos



▶ RPO and RTO on the clock (1:29)



▶ Dump, restore, verify (1:35)



▶ Point-in-time recovery with WAL (1:20)

Click a thumbnail to play on GitHub. Silent, with captions and .srt subtitles; diagrams are schematic.